KUMASI TECHNICAL UNIVERSITY

COMPUTER SCIENCE DEPARTMENT

BACK – END WEB DEVELOPMENT TECHNOLOGY

COURSE CODE: BCT 205

MERCY VICENTIA ADU GYAMFI (MRS)

APIs

- What is an API?
- Why use PHP for APIs?
- Types of API
- What is web service?
- Web Services vs APIs
- Why Use APIs?

Understanding APIs

Definition:

- An API (**Application Programming Interface**) is a set of rules that allows different software applications to communicate with each other.
- It acts as a bridge, enabling one system to send requests and receive responses from another
- Provides access to data or services

Example:

- A weather app requests data from a weather server
- The server returns temperature and conditions via an API

Types of APIs

- **Open APIs (Public APIs):** Available for external developers and users with minimal restrictions.
- **Partner APIs:** Shared externally but only with specific business partners.
- **Internal APIs (Private APIs):** Restricted to internal use within a company.
- **Composite APIs:** Combine multiple APIs into one, performing batch requests.

Understanding APIs

- **RESTful API vs SOAP API**
  - RESTful API: Uses HTTP methods, lightweight, stateless
  - SOAP API: XML-based, standardized protocol, heavier
- **HTTP Methods (GET, POST, PUT, DELETE)**
  - GET: Retrieve data
  - POST: Submit data
  - PUT: Update data
  - DELETE: Remove data
- **JSON and XML Formats**
  - JSON (JavaScript Object Notation): Lightweight, widely used
  - XML (eXtensible Markup Language): Structured, used in older APIs

Understanding APIs

- **What is a Web Service**
  - A **web service** is a standardized way for applications to communicate over the internet using a network protocol like HTTP.
  - It provides a platform for exchanging data and functionalities between systems, commonly using XML or JSON formats.

Understanding APIs

REST Web Service

- **Stands for Representational State Transfer**
  - It is an architectural style for web services.

- **Characteristics:**
  - Utilizes standard HTTP methods (GET, POST, PUT, DELETE).
  - Data is often exchanged in JSON or XML formats.
  - Scales efficiently and is lightweight.
  - Example: APIs for online shopping.

SOAP vs REST

Feature	SOAP	REST
Protocol	Strict protocol with defined rules.	Architectural style; less rigid.
Ease of Use	Complex setup and implementation.	Simpler and faster to use.
Message Format	XML only.	XML, JSON, plain text, etc.
Performance	Slower due to extensive overhead.	Faster and more lightweight.
Security	Built-in standards like WS-Security.	Relies on underlying HTTPS protocols.

API vs Web Service

Feature	API	Web Service
Definition	A set of functions allowing communication between applications.	A method of communication over a network using standard protocols.
Dependency	Can work offline (e.g., local APIs).	Requires a network for functionality.
Protocol Used	No strict protocol; supports REST, RPC, GraphQL, etc.	Typically uses SOAP or REST protocols.
Examples	Payment APIs, Social Media APIs	SOAP and RESTful web services

Understanding APIs

Why Use APIs

- APIs play a vital role in mobile app development by enabling apps to communicate with external services and systems.

- **Integration with Third-Party Services**

APIs allow mobile apps to seamlessly integrate features such as:

- Social Media: Logging in with Facebook, Twitter, or Google.
- Payment Gateways: Processing payments through APIs like PayPal or Stripe.
- Maps and Location: Embedding navigation and location services using Google Maps API.

- **Fetching and Sending Data**

Mobile apps use APIs to interact with databases or servers.

For example:

- A weather app uses APIs to fetch real-time weather data.
- An e-commerce app sends user orders to the server via APIs.

Why Use APIs

- **Cross-Platform Functionality**
  - APIs enable mobile apps to interact with backend systems, ensuring consistent functionality across iOS, Android, and web platforms.
- **Simplified Development Process**
  - Developers use APIs to avoid building complex features from scratch. For instance:
  - APIs for push notifications (like Firebase Cloud Messaging) save time and resources.
  - APIs for artificial intelligence, such as Google Vision, can bring advanced functionalities like image recognition.
- **Enhanced User Experience**
  - APIs support real-time updates and dynamic content, ensuring a smoother and more interactive experience for users.

Common APIs for Mobile App and Web Development	
Google Maps API: Enables location-based services, such as mapping, geocoding, and real-time traffic updates.	**YouTube Data API**: Enables apps to access YouTube content, such as videos, playlists, and channels.
Firebase API: Provides tools for real-time databases, authentication, cloud storage, and analytics.	**Twilio API**: Powers communication features like SMS, voice, and video calls.
Facebook Graph API: Allows integration with Facebook for user authentication, data access, and content sharing.	**Weather APIs (e.g., OpenWeatherMap)**: Provides weather data for apps requiring real-time forecasts.
Twitter API: Facilitates interaction with Twitter, including posting tweets and retrieving user data.	**Amazon S3 API**: Offers cloud storage solutions for apps needing scalable data storage.
Stripe API: Supports payment processing for e-commerce and subscription-based apps.	**Spotify API**: Allows integration with Spotify for music streaming and playlist management.

Using APIs

- **Understand the API Documentation**

  Start by reviewing the API's documentation, which includes:
  - Base URL (e.g., https://api.example.com).
  - Endpoints (e.g., /users, /posts).
  - Authentication methods (e.g., API key, OAuth).
  - Request and response formats (e.g., JSON, XML).

- **Install Required Tools**
  - Ensure you have PHP installed and consider using extensions like cURL or external libraries like Guzzle to make HTTP requests.

PHP API Interaction

Simple GET Request (Fetch Data)

```php
<?php

// API endpoint (URL)

$url = "https://jsonplaceholder.typicode.com/users";

// Fetch data from the API

$response = file_get_contents($url);

// Convert JSON response into a PHP associative array

$data = json_decode($response, true);

// Loop through each user and display their name

foreach ($data as $user) {

    echo $user['name'] . "<br>"; // Output user name

}

?>
```

GET Request Using cURL

```php
<?php

// Initialize cURL session

$ch = curl_init();

// Set the API URL

curl_setopt($ch, CURLOPT_URL, "https://jsonplaceholder.typicode.com/posts");

// Return response as a string instead of outputting it

curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);

// Execute the request

$response = curl_exec($ch);

// Close cURL session

curl_close($ch);

// Convert JSON response into PHP array

$data = json_decode($response, true);

// Display the title of the first post

echo $data[0]['title'];
```

POST Request (Send Data)

```php
<?php
// API endpoint
$url = "https://jsonplaceholder.typicode.com/posts";

// Data to send to the API
$data = [
    "title" => "My Post",
    "body" => "This is content",
    "userId" => 1
];

// Set HTTP request options
$options = [
    "http" => [
        "method" => "POST", // Specify POST method
        "header" => "Content-Type: application/json", // Send data as JSON
        "content" => json_encode($data) // Convert PHP array to JSON
    ]
];

// Create a stream context with the options
$context = stream_context_create($options);

// Send request and get response
$response = file_get_contents($url, false, $context);

// Output API response
echo $response;
?>
```

API with Authentication (API Key)

```php
<?php
// Store API key (provided by the service)
$apiKey = "YOUR_API_KEY";

// Build URL with API key as a parameter
$url = "https://api.example.com/data?api_key=" . $apiKey;

// Fetch data from API
$response = file_get_contents($url);

// Convert JSON to PHP array
$data = json_decode($response, true);

// Display the result
print_r($data);
?>
```

Handling Errors

```php
<?php
// Attempt to fetch data (the @ suppresses warnings)
$response = @file_get_contents("https://api.example.com/data");

// Check if request failed
if ($response === FALSE) {
    echo "Error fetching API"; // Display error message
} else {
    // Convert JSON response to array
    $data = json_decode($response, true);

    // Print the data
    print_r($data);
}
?>
```

Using APIs

- **Handle API Responses**

  APIs typically return data in JSON or XML format. Use PHP's functions to handle the response:

  - JSON: `json_decode($response, true)`
  - XML: `simplexml_load_string($response)`

- **Implement Error Handling**

  Always handle errors gracefully, such as:

  - Checking HTTP response codes.
  - Catching connection errors.
  - Validating the response structure

- **Optimize the Process**

  - Consider caching responses to reduce redundant requests and improve performance. For instance, use libraries like Redis or Memcached.

THANK YOU !

NO MATTER WHICH FIELD OF WORK YOU WANT TO GO IN, IT IS OF GREAT IMPORTANCE TO LEARN AT LEAST ONE PROGRAMMING LANGUAGE.